Analysis of Algorithms

Analysis of Algorithms

What is the goal?

- Analyze time requirements - predict how **running time** increases as the **size of the problem** increases:

$$\boxed{\text{time} = f(\text{size})}$$

Why is it useful?

- To compare different algorithms.

Defining “problem size”

- Typically, it is straightforward to identify the size of a problem, e.g.:
  - size of array
  - size of stack, queue, list etc.
  - vertices and edges in a graph
- But not always …

Time Analysis

- Provides upper and lower bounds of running time.

$$Lower\ Bound \leq Running\ Time \leq Upper\ Bound$$

- Different types of analysis:
  - Worst case
  - Best case
  - Average case

Worst Case

- Provides an <u>upper bound</u> on running time.
- An absolute guarantee that the algorithm would not run longer, no matter what the inputs are.

$$Lower\ Bound \leq Running\ Time \leq Upper\ Bound$$

Best Case

- Provides a <u>lower bound</u> on running time.
- Input is the one for which the algorithm runs the fastest.

$$Lower\ Bound \leq Running\ Time \leq Upper\ Bound$$

Average Case

- Provides an estimate of "average" running time.
- Assumes that the input is random.
- Useful when best/worst cases do not happen very often (i.e., few input cases lead to best/worst cases).

$$Lower\ Bound \leq Running\ Time \leq Upper\ Bound$$

Example: Searching

- Problem of *<u>searching</u> an ordered list*.
  - Given a list L of n elements that are <u>sorted</u> into a definite order (*e.g.*, numeric, alphabetical),
  - And given a particular element x,
  - Determine whether x appears in the list, and if so, return its index (i.e., position) in the list.

Linear Search

procedure *linear search*

(x: integer, $a_1, a_2, \ldots, a_n$: distinct integers)

$i := 1$

while $(i \leq n \wedge x \neq a_i)$

$\quad i := i + 1$

if $i \leq n$ **then** *location* $:= i$

else *location* $:= -1$

return *location*

NOT EFFICIENT!

How do we analyze an algorithm?

- Need to define objective measures.

(1) Compare execution times?

Not good: times are specific to a particular machine.

(2) Count the number of statements?

Not good: number of statements varies with programming language and programming style.

Example

Algorithm 1

```
arr[0] = 0;
arr[1] = 0;
arr[2] = 0;
 ...
arr[N-1] = 0;
```

Algorithm 2

```
for(i=0; i<N; i++)
  arr[i] = 0;
```

How do we analyze an algorithm? (cont.)

(3) Express running time t as a <u>function</u> of problem size n (i.e., $t=f(n)$).

- Given two algorithms having running times $f(n)$ and $g(n)$, find which functions grows faster.

- Such an analysis is <u>independent</u> of machine time, programming style, etc.

How do we find *f(n)*?

(1) Associate a "cost" with each statement.

(2) Find total number of times each statement is executed.

(3) Add up the costs.

Algorithm 1

	Cost
arr[0] = 0;	c_1
arr[1] = 0;	c_1
arr[2] = 0;	c_1
...	
arr[N-1] = 0;	c_1

$c_1+c_1+...+c_1 = c_1 \times N$

Algorithm 2

	Cost
for(i=0; i<N; i++)	c_2
arr[i] = 0;	c_1

$(N+1) \times c_2 + N \times c_1 =$

$(c_2 + c_1) \times N + c_2$

How do we find *f(n)*? (cont.)

	Cost
sum = 0;	c_1
for(i=0; i<N; i++)	c_2
for(j=0; j<N; j++)	c_2
sum += arr[i][j];	c_3

$$c_1 + c_2 \times (N+1) + c_2 \times N \times (N+1) + c_3 \times N \times N$$

Comparing algorithms

- Given two algorithms having running times *f(n)* and *g(n)*, how do we decide which one is faster?

- Compare "rates of growth" of *f(n)* and *g(n)*

Understanding Rate of Growth

- Consider the example of buying *elephants* and *goldfish:*

Cost: (cost_of_elephants) + (cost_of_goldfish)

Approximation:

Cost ~ cost_of_elephants

Understanding Rate of Growth (cont'd)

- The low order terms of a function are relatively insignificant for **large** n

$$n^4 + 100n^2 + 10n + 50$$

Approximation:

$$n^4$$

- Highest order term determines rate of growth!

Example

- Suppose you are designing a website to process user data (*e.g.*, financial records).
- Suppose program A takes $\boldsymbol{f_A(n)=30n+8}$ microseconds to process any n records, while program B takes $\boldsymbol{f_B(n)=n^2+1}$ microseconds to process the n records.
- Which program would you choose, knowing you'll want to support millions of users?

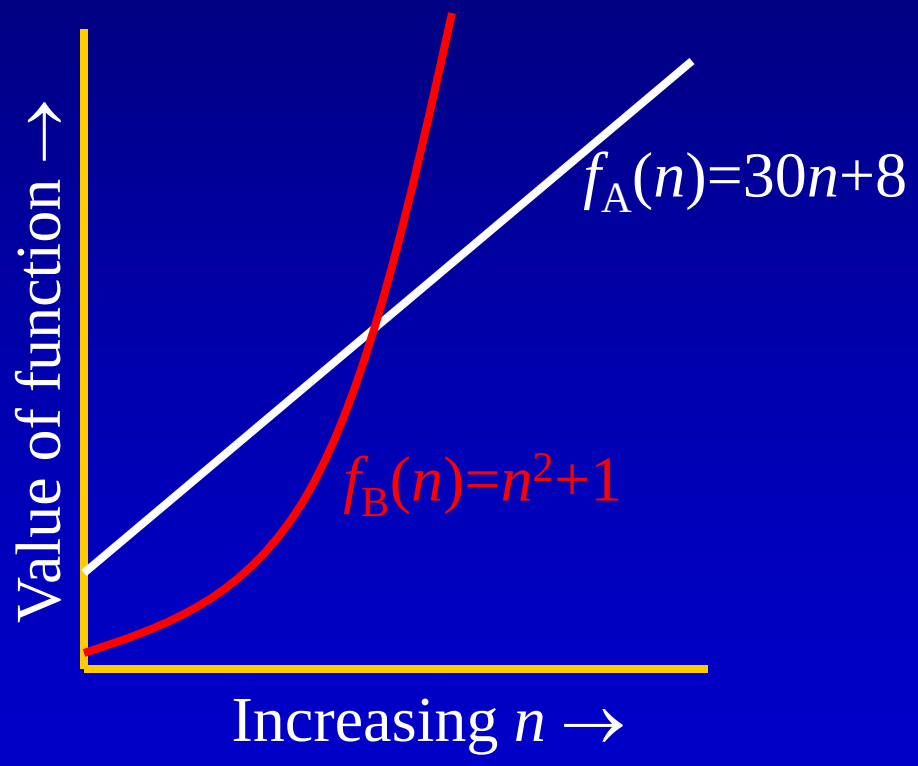
Compare rates of growth:

$\mathbf{30n+8 \sim n}$ **and** $\mathbf{n^2+1 \sim n^2}$

A

Visualizing Orders of Growth

- On a graph, as you go to the right, a faster growing function eventually becomes larger...



Rate of Growth ≡Asymptotic Analysis

- Using *rate of growth* as a measure to compare different functions implies comparing them **asymptotically** (i.e., as n → ∞)

- If $f(x)$ is *faster growing* than $g(x)$, then $f(x)$ always eventually becomes larger than $g(x)$ **in the limit** (i.e., for large enough values of x).

Asymptotic Notation

- **O notation:** asymptotic "less than":

$f(n)=O(g(n))$ <u>implies:</u> $f(n)$ "$\leq$" $c\ g(n)$ in the limit*

c is a constant

*formal definition in CS477/677

Asymptotic Notation

- **Ω notation:** asymptotic "greater than":

$f(n) = \Omega(g(n))$ <u>implies:</u> $f(n)$ "$\geq$" $c\, g(n)$ in the limit*

c is a constant

*formal definition in CS477/677

Asymptotic Notation

- **Θ notation:** asymptotic "equality":

$f(n) = \Theta(g(n))$ <u>implies:</u> $f(n)$ "=" $c\, g(n)$ in the limit*

c is a constant

(provides a tight bound of running time)
(best and worst cases are same)

*formal definition in CS477/677

Big-O Notation - Examples

$f_A(n)=30n+8$ is $O(n)$

$f_B(n)=n^2+1$ is $O(n^2)$

$10n^3 + 2n^2$ is $O(n^3)$

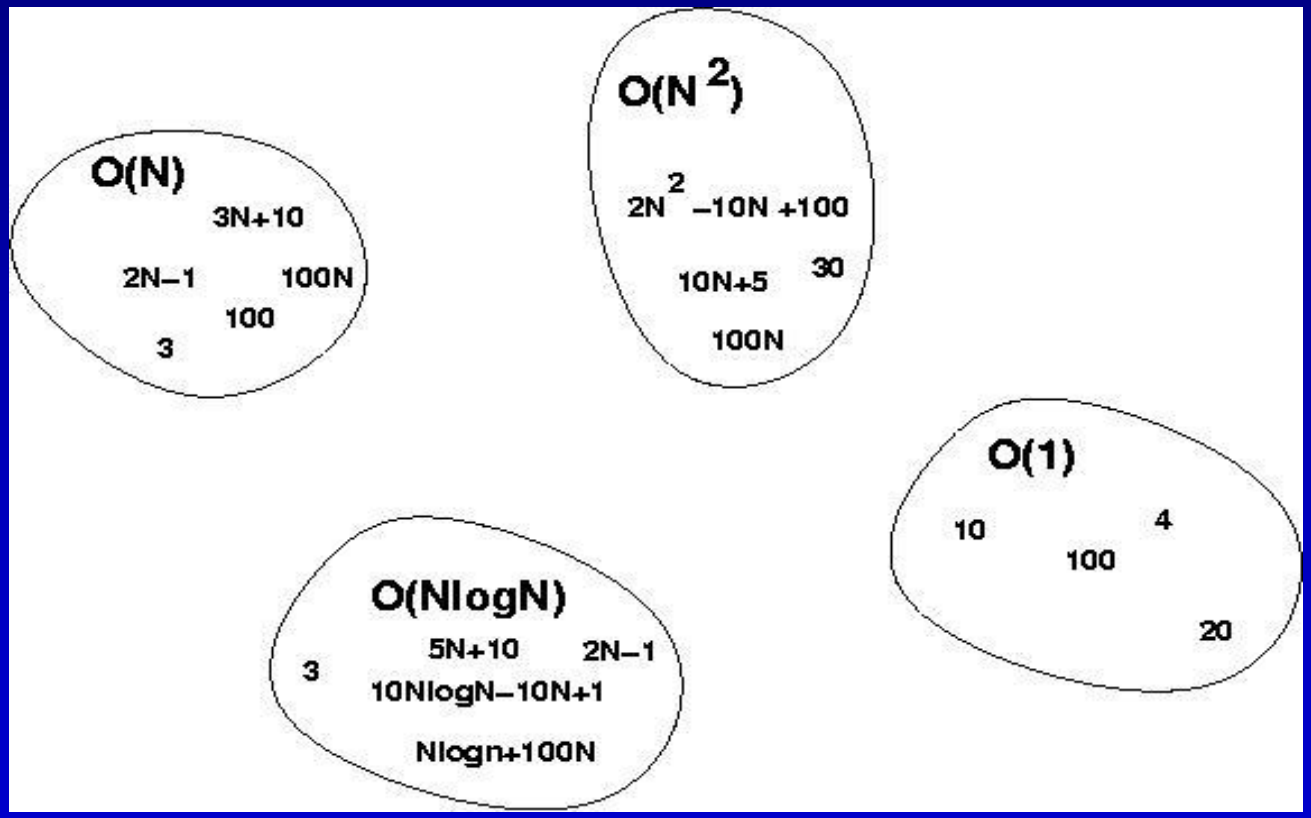
$n^3 - n^2$ is $O(n^3)$

1273 is $O(1)$

More on big-O

O(g(n)) is a set of functions f(n)

f(n) ϵ O(g(n)) if **"f(n)≤cg(n)"**



Big-O Notation - Examples

$f_A(n)=30n+8$ is $O(n)$ or $O(n^2)$

$f_B(n)=n^2+1$ is $O(n^2)$ or $O(n^4)$

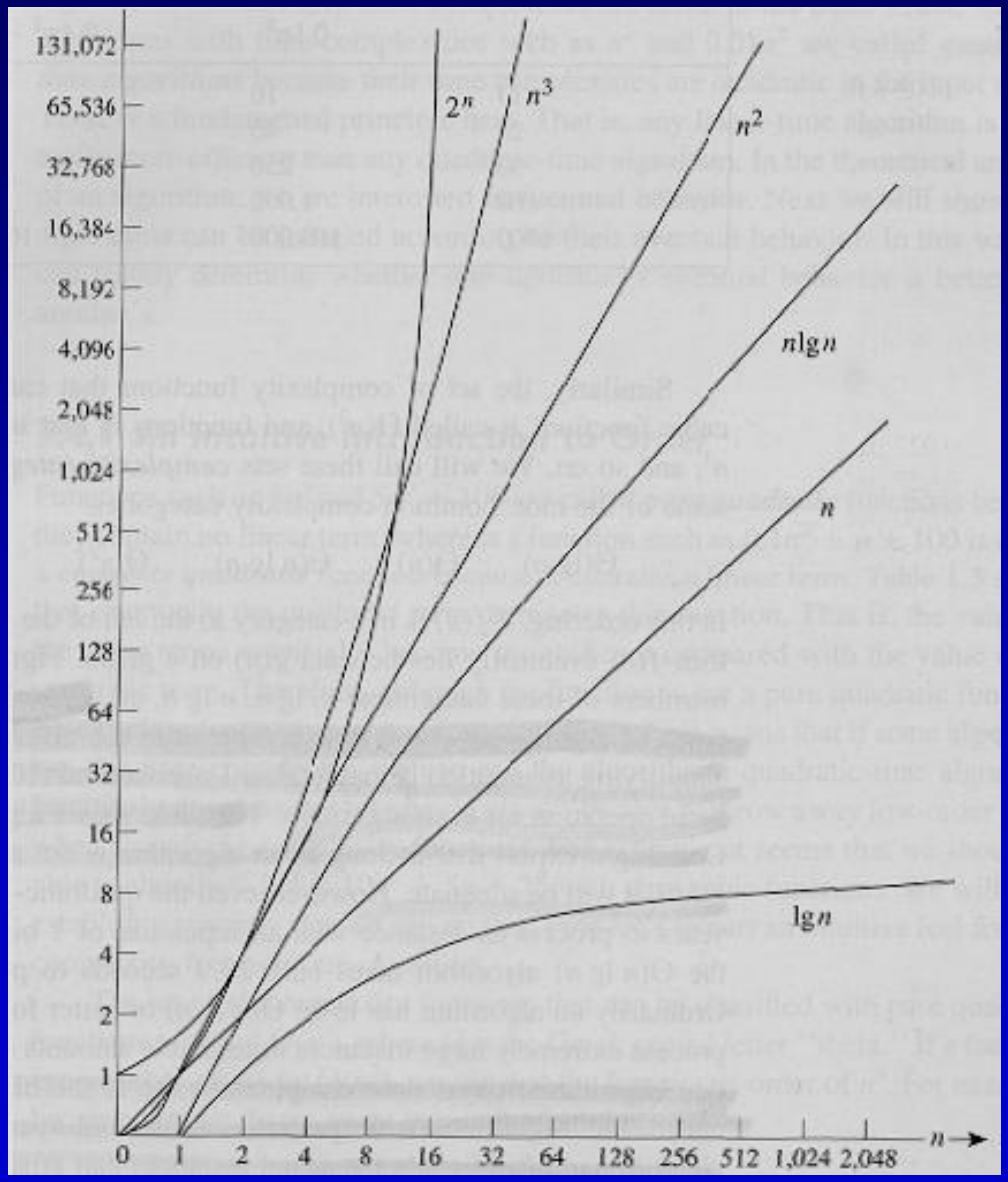
$10n^3 + 2n^2$ is $O(n^3)$ or $O(n^4)$

$n^3 - n^2$ is $O(n^3)$ or $O(n^5)$

1273 is $O(1)$ or $O(n)$

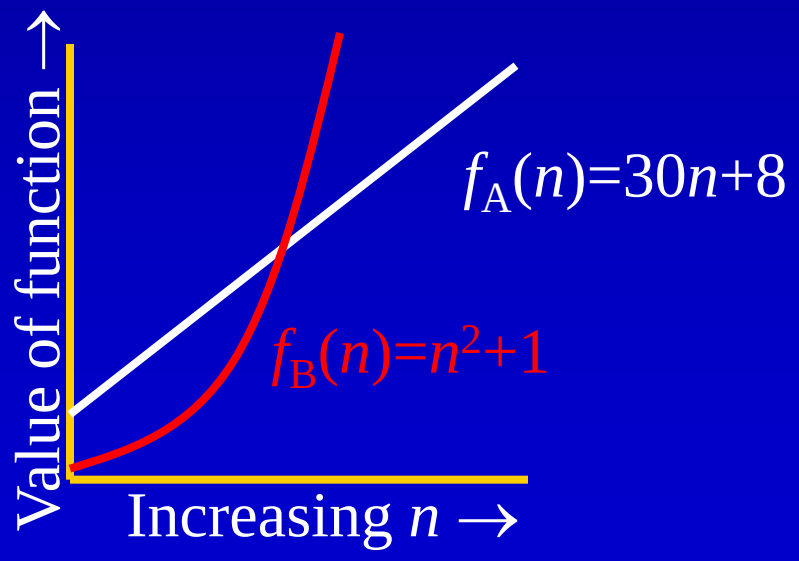
But it is important to use as “tight” bounds as possible!

Common orders of magnitude



Algorithm speed vs function growth

- An O(n^2) algorithm will be <u>slower</u> than an O(n) algorithm (for large n).
- But an O(n^2) function will grow <u>faster</u> than an O(n) function.



Estimating running time

Algorithm 1

	Cost
arr[0] = 0;	c1
arr[1] = 0;	c1
arr[2] = 0;	c1
...	
arr[N-1] = 0;	c1

---

c1+c1+...+c1 = c1 x N

Algorithm 2

	Cost
for(i=0; i<N; i++)	c2
arr[i] = 0;	c1

(N+1) x c2 + N x c1 =
(c2 + c1) x N + c2

O(N)

Estimate running time (cont.)

	Cost
sum = 0;	c1
for(i=0; i<N; i++)	c2
for(j=0; j<N; j++)	c2
sum += arr[i][j];	c3

$$c1 + c2 \times (N+1) + c2 \times N \times (N+1) + c3 \times N \times N$$

$$O(N^2)$$

Examples

```
i = 0;
while (i<N) {
   X=X+Y;                    // O(1)
   result = mystery(X);      // O(N), just an example...
   i++;                      // O(1)
}
```

- The body of the while loop: O(N)
- Loop is executed: <u>N times</u>

$$N \times O(N) = O(N^2)$$

Examples (cont.'d)

```
if (i<j)
   for ( i=0; i<N; i++ )
        X = X+i;
else
   X=0;
```

O(N)

O(1)

Max (O(N), O(1)) = O (N)

Examples (cont.'d)

Rank the following **functions** by order of growth from the slowest to the fastest (notation: $\lg n = \log_2 n$)

$$1000,\ 10\lg n,\ 4n^2,\ \ n^2,\ \ 2^n,\ \ 100n,\ \ 2^{\lg n}$$

$$1000 \quad 10\lg n \quad 2^{\lg n} \quad 100n \quad n^2 \quad 4n^2 \quad 2^n$$

(Slowest → Fastest)

Examples (cont.'d)

Compare the two functions n^2 and $2^n/4$ for various values of n. Determine when the second becomes larger than the first.

$n^2 \geq 2^n/4$ for $n \leq 8$

So, $2^n/4$ becomes larger for $n \geq 9$

Examples (cont.'d)

- Analyze the complexity of the following code segments:

```
(a) sum = 0;
    for(i=1; i<=2*n; i++)
      sum = sum + 1;

(b) sum = 0;
    for(i=1; i<=n*n; i++)
      sum = sum + 1;

(c) sum=0;
    for(i=1; i<=n; i++)
      sum = sum + n;

(d) sum = 0;
    for(i=1; i<=n i++)
      for(j=1; j<=i; j++)
        sum = sum + i;

(e) sum = 0;
    for(i=1; i<=100; i++)
      for(j=1; j<=n; j++)
        sum = sum + i;

(f) sum = 0;
    for(i=1; i<=n; i++)
      for(j=1; j<=n; j*=2)
        sum = sum + 1;
```

$$S_n = \frac{n}{2}(a_1 + a_n)$$

Examples (cont.'d)

- Analyze the complexity of the following code segments:

(a) sum = 0; $O(1)$

$O(2n)$ for(i=1; i<=2*n; i++)

$O(1)$ → sum = sum + 1;

Total: $O(2n) = O(n)$

(b) sum = 0; $O(1)$

$O(n^2)$ for(i=1; i<=n*n; i++)

$O(1)$ → sum = sum + 1;

Total: $O(n^2)$

(c) sum=0; $O(1)$

$O(n)$ for(i=1; i<=n; i++)

$O(1)$ → sum = sum + n;

Total: $O(n)$

(d) sum = 0; $O(1)$

$O(n)$ for(i=1; i<=n i++)

$O(i)$ for(j=1; j<=i; j++)

$O(1)$ sum = sum + i;

when $i=1$ $j \le 1$ 1 time

$i=2$ $j \le 2$ 2 times

$\vdots$

$i=n$ $j \le n$ n times

Total $1+2+\cdots+n = \frac{n(n+1)}{2} = O(n^2)$

(e) sum = 0; $O(1)$

$O(1)$ for(i=1; i<=100; i++)

$O(n)$ for(j=1; j<=n; j++)

$O(1)$ sum = sum + i;

Total: $O(n)$

(f) sum = 0; $O(1)$

for(i=1; i<=n; i++) $O(n)$

for(j=1; j<=n; j*=2) $O(\lg n)$

sum = sum + 1; $O(1)$

Total: $O(n \lg n)$